

Virtual Lab — 2026-07-01 — Making the Flat-SHAP Baseline a Fair Opponent

Why this note exists

The student's plan compares two ways of scoring genes: a grouped, two-step method (score the modules, then split each module's score among its genes) against plain ungrouped SHAP over all genes at once. The plan gives both the same 1,000 "probes." That makes the comparison unfair, and this note explains what a probe is, why 1,000 is unfair, and exactly how to make the two sides comparable.

Keep this handy: the fix lives in the *methods*, and it is needed under every version of the research question — even a neutral "where did the credit land" comparison is only trustworthy if both methods were fed comparably.

What a "probe" is

KernelSHAP works out each gene's credit by trying many random on/off combinations of genes and watching how the model's prediction shifts each time. Each combination it tries is one probe — the SHAP library's own term is a *coalition sample*. From a pile of "this combination was on, the prediction moved this much" pairs, it solves a small regression, and the answers are the per-gene credits.

The key fact: it is solving for **one credit number per gene**, so the amount of evidence it needs grows with the number of genes. Solving for 5,000 numbers takes far more probes than solving for 40.

Why 1,000 probes is unfair here

- The **grouped** run solves for a few dozen module numbers. Call it ~40 modules. 1,000 probes over 40 modules is ~25 probes of evidence behind each. Richly fed.
- The **flat** or ungrouped run solves for ~5,000 gene numbers. 1,000 probes over 5,000 genes is fewer probes than unknowns. Starved — most gene credits are really guesses that the solver papers over with its built-in regularization.

So if the grouped method then "wins," you cannot tell whether it won because grouping recovers biology or simply because the flat side was starved and the grouped side was well-fed. The race is rigged before it starts.

The fix: feed each gene as well as each module is fed

"Give flat SHAP enough probes" means raising the flat probe count until each **gene** is backed by about as much evidence as each **module** is in the grouped run.

Starting arithmetic:

```
flat probes ≈ grouped probes × (genes ÷ modules)
            ≈ 1,000 × (5,000 ÷ 40)
            ≈ 125,000
```

That is the honest order of magnitude — "raise it from one thousand to somewhere past a hundred thousand," not "nudge it up." Treat 125,000 as where to *start looking*, not as a law; the two games are not perfectly comparable, so the number is a guess to be checked, not trusted.

Two practical details

The knob. In the SHAP library this is just the sample-count setting you pass when you explain a prediction. In the classic `KernelExplainer` it is `nsamples`; in the newer unified `Explainer` it is `max_evals`. Confirm the exact argument name against whichever SHAP version you pin — this detail drifts between versions. Worth knowing: left on its default, the library already scales this with the feature count, so the plan's hard-coded 1,000 actually *overrides* a scaling the tool would otherwise have done for you.

Cost is not a problem on this dataset. The models are small and the background is a single mean vector, so each probe is one quick model call. Even 125,000 probes across all 68 plants and all three models is minutes of compute on free Colab, not hours. So "match the budget" is a real, affordable option here — you do not have to retreat to shrinking the gene set or demoting the baseline unless the check below surprises you.

The recipe

1. Pick a matched target.

flat probes $\approx$ grouped probes $\times$ (genes $\div$ modules). With $\sim 1,000$ probes over ~ 40 modules and $\sim 5,000$ genes, that is $\sim 125,000$.

2. Sweep and check.

Run flat SHAP at a few rising budgets — say 1,000, then 10,000, then 100,000 — and watch the top-gene ranking. When adding more probes stops changing which genes come out on top, it has had enough. That convergence check is what actually earns the word "fair"; the arithmetic just tells you where to start looking.

3. Use the settling point.

Take the smallest budget where the ranking has stabilized, and report that number next to the grouped run's, so anyone can see both methods were fed comparably.